REINFORCEMENT LEARNING FOR STOCK MARKET PORTFOLIO MANAGEMENT

BY

OJETOKUN OLUWAFEMI AKINWALE

(190805019)

SUBMITTED TO

THE DEPARTMENT OF COMPUTER SCIENCES,

FACULTY OF COMPUTING AND INFORMATICS,

UNIVERSITY OF LAGOS, NIGERIA

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MARCH 2026

SUPERVISOR: DR. B. A. SAWYERR

TABLE OF CONTENTS

CERTIFICATION

This is to certify that this project titled **"REINFORCEMENT LEARNING FOR STOCK MARKET PORTFOLIO MANAGEMENT"** was carried out by **OJETOKUN OLUWAFEMI AKINWALE** with Matriculation Number **190805019**, and submitted to the Department of Computer Sciences, Faculty of Computing and Informatics, University of Lagos, in partial fulfillment of the requirements for the award of Bachelor of Science (B.Sc.) degree in Computer Science.

_____ _____

DR. B. A. SAWYERR **DATE**

SUPERVISOR

_____ _____

DR. C. O. YINKA-BANJO **DATE**

HEAD OF DEPARTMENT

ABSTRACT

Stock market portfolio management attempts to invest capital in a strategic manner to achieve maximum risk-adjusted returns. The classical models, including the Mean Variance Optimization and the Capital Asset Pricing Model, are based on the strict and unrealistic assumptions about market efficiency and distribution of returns. These standard model methods often do not work with live, non-stationary financial markets, and so provide suboptimal performance when faced with real-world transaction costs or when faced with sudden regime changes.

To overcome these shortcomings, this paper examines various model-free Deep Reinforcement Learning (DRL) techniques. Unlike the traditional frameworks which require explicit market forecasting, model-free DRL agents learn the best trading strategies by trial-and-error interactions with past data. This study applies and comparatively evaluates two state-of-the-art DRL systems: Double Deep Q-Networks (DDQN) and Proximal Policy Optimization (PPO).

The methodology employed in this project is based on the Machine Learning Development Lifecycle (MLDL) and mathematically models the trading problem as a Markov Decision Process (MDP). The research environment adopted is a self-built trading simulator based on the OpenAI Gymnasium framework.

This study shows the real-world feasibility of model-free reinforcement learning with realistic market constraints, and provides valuable information on automated trading systems to both scholars and practitioners in the industry.

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Portfolio management is the process of initial selection of a group of investments (stocks, bonds, cash, etc) and the subsequent rebalancing of their weights periodically in order to attain predetermined investment goals. These goals differ among individuals and entities, but they typically converge on maximizing returns and minimizing risk (Jiang et al., 2017; Markowitz, 1952). The specific objective function chosen depends on factors like risk tolerance and risk preference.

Due to factors that cause the nature of live markets to be largely unpredictable, it is advised to follow strategies that adapt smartly and respond to abrupt market changes while capitalizing on asset profitability (Mosavi et al., 2020).

Traditional portfolio management techniques exist (Markowitz, 1952), but they mostly make strong assumptions on market conditions and behavior, and fail to efficiently account for the dynamic, non-stationary nature of markets. When battletested in real life environments, these traditional techniques produce suboptimal results (Mosavi et al., 2020).

The introduction of machine learning as an alternative stock market portfolio management technique offers adaptive learning from market interactions. Reinforcement learning enables agents to learn decision-making policies that are optimal by repeatedly interacting directly with

an environment, sometimes without requiring explicit models of the underlying system dynamics (Sutton & Barto, 2018). The agents can then formulate and utilize effective strategies through a feedback loop of observing the market (states), adjusting the portfolio (actions) and receiving feedback in the form of the decided performance metrics such as cumulative returns, Sharpe ratio, and drawdown (rewards) (Filos, 2019; Jiang et al., 2017).

With time, the agent learns to utilize the allocation strategies with the most promise of yield and long-term returns, regardless of market condition.

1.2 Statement of the Problem

Stock market portfolio management faces some underlying challenges that limit the efficiency of the traditional optimisation processes. Classical techniques like mean-variance optimisation (Markowitz, 1952) and the Capital Asset Pricing Model make overly restrictive assumptions about the distribution of returns, market efficiency and stationarity which are systematically violated in actual financial markets. It has been shown that the financial returns are fat tailed, volatile, and non-stationary; these are properties that are simply inconsistent with the assumptions of the traditional portfolio theory (Jiang et al., 2017; Mosavi et al., 2020).

The challenges are especially harsh in the case of the dynamic and non-stationary nature of financial markets. The market environment is constantly changing based on macroeconomic changes, regulation changes, and changing investor behaviour, but traditional methods usually use rolling-window estimation methods which assume that the most recent past history can provide accurate information about future performance. This strategy proves to be disastrous in

periods of structural discontinuity and regime changes, providing strategies that are ill-suited to new market conditions (Mosavi et al., 2020). Furthermore, with increasing size of the portfolio, the traditional approaches face what is called the curse of dimensionality; the number of parameters to estimate rises quadratically with the number of assets, and the resulting estimation errors can overwhelm optimization results.

These challenges are further compounded by transaction costs and market frictions. Academic portfolio optimization tends to ignore the reality of practical trading, and makes the assumption of frictionless rebalancing. In reality, the returns on a portfolio are significantly diminished by transaction costs, which include the commission charged by the broker, the bid-ask spread, and the price impact. Traditional approaches usually deal with these costs by considering them as ad-hoc restrictions as opposed to making them part and parcel of the optimization structure (Jiang et al., 2017).

Although machine learning has experienced increased use in the financial world, the current methods have major drawbacks. Predicting asset prices using supervised learning is available in many applications, but price prediction and portfolio optimization are two entirely different tasks (Filos, 2019). A model that forecasts returns well might not be an optimal model for the portfolio because it involves risk, diversification, transaction costs and dynamics of sequential decision making. Also, labelled training data is required in supervised learning, whereas a “correct” portfolio allocation is not directly observable: it relies on future results that are not known at the time of making a decision.

Current applications of reinforcement learning in portfolio management, despite being potentially promising, are not systematically compared between algorithms in the same conditions of the experiment. Various researchers use different data sets, metrics of measurements, and comparison of baselines making it hard to form comprehensive conclusions about comparative efficiency (Mosavi et al., 2020). A large part of the literature is based on generalized conditions that neglect transaction costs, assume perfect liquidity and allow frictionless rebalancing, simplifications that lower the level of practical applicability and cast doubt on the viability in the real world (Jiang et al., 2017).

To counter these limitations, this research uses and systematically compares two model-free deep reinforcement learning algorithms: Double Deep Q -networks (DDQN) and Proximal Policy Optimization (PPO) to manage a stock portfolio under realistic trading conditions. To be more specific, the research empirically compares rigorously value-based (DDQN) and policy-gradient (PPO) algorithms in the same experimental framework with explicit transaction cost modelling, performs extensive robustness analysis under different market conditions and cost situations, and compares the performance with the known traditional strategies on multiple financial metrics with statistical significance tests. By achieving these goals, the study not only provides theoretical implications of the applicability of various paradigms of reinforcement learning to the field of finance but also offers a practical way of implementing automated portfolio management systems.

1.3 Aim and Objectives of the Study

1.3.1 Aim

The aim of this project is to examine and implement various reinforcement learning techniques for optimal stock market portfolio management, and compare each agent's performance in live market conditions to that of other agents as well as traditional techniques.

1.3.2 Objectives

The following are the objectives of this work:

1. To model the portfolio management problem as a Markov Decision Process (MDP) and attempt to solve it with the help of reinforcement learning agents.
2. To undertake portfolio optimization through several reinforcement learning algorithms.
3. To compare the performance of the reinforcement learning agent with the conventional portfolio management strategies through the Sharpe ratio, cumulative returns, and maximum drawdown.
4. To examine the acquired trading strategies and give insights to the behavior of agents and the pattern of decision making.
5. To evaluate the strength of the models observed under various market conditions and time.

1.4 Scope of the Study

This study is based on the management of an equity portfolio using a universe composed of 30 large-market-cap technology stocks sampled by the S&P 500 index. The asset selection criteria

focuses on companies that have a market capitalization of at least 50 billion USD, daily trade volume of more than 1 million shares, and full historical information from January 1, 2015 to December 31, 2024 without any gap. The key choices of major assets are Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA), Alphabet (GOOGL), Amazon (AMZN), and Meta Platforms (META) and other major technological firms.

The study applies and compares two model-free deep reinforcement learning algorithms, in-depth, Double Deep Q-Networks (DDQN) and Proximal Policy Optimization (PPO). This two-algorithm design allows value-based methods and policy gradient methods to be compared, discrete and continuous action representations to be compared, and off-policy and on-policy learning methods to be compared.

Market data is obtained using Yahoo finance through the yfinance package, over a 10-year period between the first and the last trading days of January 1, 2015 to December 31, 2024 (approximately 2,516 trading days). The data set covers a wide range of different market environments such as bull markets, bear markets, the COVID-19 crash of March 2020, and recovery periods. The data is divided into training (2015-2020, 60%), validation (2021-2022, 20%), and testing (2023-2024, 20) sets and not shuffled to maintain time dependencies and provide strict out-of-sample testing.

The implementation codebase is based on Python 3.10 with OpenAI Gymnasium (v0.29.1) to provide the standardized interface of the reinforcement learning environment, Stable-Baselines3 (v2.2.1) to find optimized implementations of the algorithm, and PyTorch (v2.1.0) to build neural networks.

The study identifies other asset classes, short selling and leverage approaches, high-frequency trading, market microstructure impacts, and live deployment factors. The study aims at backtesting and algorithm design in a controlled experimental setting to allow rigorous comparison by science.

1.5 Methodology

This work is implemented using the machine learning development lifecycle methodology, an iterative, data-driven methodology characterised by the following stages:

1. Problem definition and formulating the problem as a Markov Decision Process(MDP)
2. Data collection and preparation
3. Data cleaning and preprocessing
4. Model selection: Selection and justification of selected reinforcement learning algorithms
5. Model development: Network architecture design and hyperparameter tuning
6. Model training
7. Model evaluation
8. Results analysis and interpretation

1.6 Project Outline

This project is divided into five chapters, including this introductory chapter. Chapter two is a review of pertinent literature on portfolio management, Markov Decision Processes, machine

learning, deep learning, reinforcement learning, and deep reinforcement learning referenced in this project and their applications in the stock market portfolio management field. Chapter three describes the research methodology, the problem statement, data gathering, pre-processing and execution methods, and analysis for the implementation of this work. Chapter 4 documents the implementation, network architectures, training procedures, and hyperparameter tuning of the agents. The final chapter concludes research findings, discusses limitations, and proposes possible further research directions.

CHAPTER TWO

LITERATURE REVIEW

2.1 Portfolio Management

As described in the previous chapter, portfolio management is defined as the methodical process of choosing a set of assets (like stocks, bonds, cash, and so on) and rebalancing the weight of these assets on a regular basis to meet certain investment goals (Markowitz, 1952; Jiang et al., 2017). Markowitz (1952) proposed the basis of this issue with his Mean-Variance Optimisation (MVO) model in which portfolio selection was formalised as a trade-off between expected return and variance. Continuing on this, the Capital Asset Pricing Model (CAPM) provided a single factor equilibrium pricing model between the expected returns and the systematic risk. These classical schools of thought were used in financial theory over decades and are still used in classroom curricula.

However, both MVO and CAPM are based on restrictive assumptions that are systematically broken in the actual financial markets. They make the assumption that the returns on assets are stationary, normally distributed, the markets are frictionless (no transaction costs, perfect liquidity), and that the investors have the same expectations (Jiang et al., 2017; Mosavi et al., 2020). It has long been empirically demonstrated that financial returns follow fat tails, volatility clustering and non-stationarity which are all properties that are fundamentally incompatible with these assumptions. In practice, when classical portfolio strategies are applied in real trading conditions, they often yield suboptimal outcomes, especially in times of structural change in the

market like a financial crisis or a regulatory change (Mosavi et al., 2020). The shortcomings of conventional approaches have encouraged the use of machine learning processes to portfolio management. Asset price prediction has been widely applied to supervised learning approaches, such as the ones mentioned above (Mosavi et al., 2020). Although these models are capable of making predictions of returns with reasonable accuracy, it is a fact that price prediction and portfolio optimization are two completely different things (Filos, 2019). The fact that a model predicts good returns does not imply that effective allocation will lead to good portfolios since factors like risk, diversification, transaction costs and the sequential character of decisions made must be included in the allocation. Moreover, supervised learning needs labelled training data, whereas a correct portfolio allocation cannot be observed directly and must rely on future results, which are not known at the moment of making the decision.

These weaknesses indicate that a decision making framework is required that is able to learn adaptive strategies directly out of interactions in the market, manage sequential decisions in the face of uncertainty and integrate realistic trading constraints directly into the optimisation process itself. The mathematical framework of Markov Decision Process (MDP), which is discussed in the following section, offers just such a mathematical framework.

2.2 Markov Decision Processes

A Markov Decision Process (MDP) is a mathematical model that is used to model sequential decision making in stochastic environments. The property that defines it is the Markov Property,

according to which the probability of going to any given future state only depends on the present state and the action being performed, but not on the history of the previous states.

An MDP is formally described as a five-tuple (S, A, P, R, γ), where:

S – Set of possible states for the agent

A – Set of possible actions the agent can take (this is independent of the state)

P – Transition probability function

R – Reward function, the cost or reward associated with an action in a state

γ – Discount factor (controls how much future rewards matter)

The portfolio management problem can be mapped onto the MDP framework. The agent (portfolio manager) at every time step observes a state which includes market information like past prices, technical indicators and the current portfolio weights. It then makes a decision by choosing new diversification of the portfolio in the available assets. The environment changes to a new state based on the market movements on the next day (or the next period), and the agent gets a reward based on a performance measure that is selected, e.g., the portfolio return or the Sharpe ratio (Jiang et al., 2017; Filos, 2019). This formulation has the critical characteristics of portfolio management, i.e. sequential decisions, stochastic results, and delayed effects of actions.

Since the portfolio management problem may be considered as an MDP, the obvious question is how to determine the optimal policy, i.e. the function of states to actions that optimises the

expected cumulative discounted reward. Reinforcement learning, which is discussed below, offers a family of algorithms that is specifically targeted at this purpose.

2.3 Machine Learning

Machine learning can be described as a field of artificial intelligence that employs algorithms that are trained to mimic the learning capabilities of humans, thus providing them the ability to predict outcomes from new unexploited data without human interaction (Coursera, 2024, para. 1). The three most commonly used major categories of machine learning are supervised learning, unsupervised learning and reinforcement learning.

Supervised learning entails first labeling a structured dataset, then training an algorithm using the dataset to learn a mapping that can predict results on new, unseen data. This type of machine learning is majorly suited for classification and regression models.

Unsupervised learning, in contrast to supervised learning, works with large unlabeled datasets and leaves it to the algorithm to discover structures and relationships itself. Agents built using this machine learning technique tend to excel at discovering unforeseen associations between large volumes of data. However, it may be challenging to evaluate the agents' performance because it does not have predefined input-output dataset pairs, as is the case with supervised learning.

Reinforcement learning is a subfield of machine learning in which an agent, via repeated cycles of interaction with the surrounding environment, and feedback, learns to make optimal decisions

under varying circumstances. Reinforcement learning is a form of learning that uses trial-and-error to maximise cumulative return.

2.4 Reinforcement Learning

Reinforcement learning (RL) is a branch of machine learning where an agent learns to make best decisions by repeatedly interacting with an environment by trial and error and receiving feedback in the form of reward signals (Sutton and Barto, 2018). In contrast to supervised learning, which involves the use of labelled input-output pairs, RL works by the principle of evaluative feedback: the agent receives the feedback of the quality of an action in the form of a reward, but not the feedback about the optimal action. This characteristic renders RL especially appropriate to the problem where there is no labelled information available or the optimal solution of the problem is not known beforehand, which is the case in portfolio management.

The RL algorithms may be further split into two paradigms: model-based and model-free. Model-based reinforcement learning (MBRL) refers to the agent building an internal predictive model of the environment dynamics, that is, learning the transition probabilities and the reward function, and subsequently planning the optimal actions based on the model (Deisenroth and Rasmussen, 2011). Although MBRL can be extremely sample-efficient (in essence, needing fewer iterative loops with the environment to learn a strong policy), its performance inherently hinges on the precision of the learned model.

In the case where the model is not accurate, the agent will optimize over inaccurate simulations and this will cause compounding errors and poor performance in the real-world. This weakness

is called model bias, and is especially bad in financial markets where the underlying dynamics themselves are stochastic, non-stationary and not practically modelable.

By comparison, model-free reinforcement learning does not model the environment. It is rather self-taught to act optimally by simply using experience to estimate the value of actions (value-based methods), or the optimal policy itself (policy gradient methods) based on observed transitions and rewards (Sutton and Barto, 2018). This paradigm is the logical option of portfolio management: since the financial markets are notorious in being hard to model (a model-based agent would break down whenever its market simulation does not match the reality), a model-free agent avoids this issue altogether and is interested in finding out what portfolio allocations would provide the most optimal risk-adjusted returns in the observed market environment (Jiang et al., 2017; Filos, 2019; Mosavi et al., 2020).

2.5 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) is the combination of deep learning models with the concepts of reinforcement learning, through which agents are able to learn the best possible policy by directly working with high-dimensional sensory data, without any need of feature engineering (Mnih et al., 2015). This amalgamation is a solution to a basic weakness of classical reinforcement learning; the impossibility of scaling to large-state space problems. DRL algorithms can automatically learn hierarchical representations of raw data by using deep neural networks as function approximators and they can also optimize decision-making policies (Arulkumaran et al., 2017).

DRL has been successful because it can deal with the “curse of dimensionality” which afflicted the original tabular RL approaches. In contrast to classical Q-learning, where it is necessary to explicitly maintain value estimates in each state-action pair - which is computationally infeasible in high-dimensional settings - deep neural networks can extrapolate similar states by learned feature representations (Francois-Lavet et al., 2018). This ability has enabled breakthroughs in areas such as game playing (Silver et al., 2016) to robotic control (Lillicrap et al., 2015) and, most importantly, financial portfolio management (Jiang et al., 2017).

There are three major families of deep reinforcement learning algorithms, which can be classified in terms of their optimization strategy: value-based methods, policy gradient methods, and actor-critic methods. The categories have unique benefits and drawbacks of sample efficiency, stability, and continuous or discrete action space applicability.

Value-Based Methods

Value-based techniques approximate the optimal action-value function, Q(s, a), which is the cumulative reward expected to take action, a, in state, s, and then use the optimal policy thereafter (Sutton and Barto, 2018). After learning this value function, the optimum policy can be obtained by choosing the action with the maximum estimated value in every state. This method proves to be especially useful in discrete action spaces when exhaustive action analysis is computationally possible.

The Deep Q-Network (DQN) algorithm, which was proposed by Mnih et al. (2015) in their seminal article, Human-level Control through Deep Reinforcement Learning, is a breakthrough

in the study of reinforcement learning. DQN was able to implement Q-learning with deep Convolutional Neural Networks as the first algorithm to demonstrate that an algorithm with fixed hyperparameters could be trained to play 49 different Atari 2600 games directly with raw pixel inputs and perform well, on par with professional human game testers (Mnih et al., 2015).

DQN solves two key problems that have hitherto hindered stable convergence in the case of neural networks being used to implement Q-learning: correlation between sequential observations and the non-stationarity of the optimization target. DQN proposed two major innovations to address these challenges, which have become common in deep RL (Mnih et al., 2015):

Experience Replay: As opposed to learning based on sequential experiences as they unfold, DQN maintains a replay buffer of transitions (s_t, a_t, r_t, s_{t+1}) and picks random mini-batches in training. This method separates time-correlation between training samples, which greatly enhances the learning stability and data efficiency since the same experience can be reused many times (Mnih et al., 2015). The replay buffer is a finite capacity cache, which generally contains the last one million transitions.

Fixed Target Network: To overcome the issue of non-stationary targets when updating the Q-learning, DQN has an additional target network Q with parameters that are periodically updated in tandem with the online network parameters, θ. The online network is trained through gradient descent, and the target network produces the Q-value targets that are used in the loss (Mnih et al., 2015). This decoupling stops the bad feedback loop in which changes in Q-values would instantaneously modify the target values to be used in subsequent changes, thus stabilizing training.

DQN loss is defined as an average squared error between the predicted and target Q-values:

$$L(\theta) = [(r_t + \gamma \max_{a'} Q(s_{t+1,} a'; \theta^{-}) - Q(s_{t,} a_t; \theta))^2] \quad (1)$$

with ɣ being the discount factor, $Q(s_{t,} a_t; \theta)$ being the forecasted Q-value of the online network, and $Q(s_{t+1,} a'; \theta^{-})$, being the estimate of the Q-value of the maximum value of the target network of the next state (Mnih et al., 2015).

Although DQN is a groundbreaking success, it is inherently restricted to discrete action spaces, since it needs to calculate $\max_{a'} Q(s', a')$ on all actions. This limitation renders vanilla DQN inapplicable to the portfolio management problem where actions are continuous portfolio weight values. Nevertheless, the architectural principles of DQN, especially experience replay and target networks, have been implemented and modified in later algorithms that are intended to be used in continuous control.

Policy Gradient Methods

The policy gradient approach represents an entirely different approach where the policy p *(π(a|s; θ))* is explicitly parameterized and optimized, instead of learning a value function based on which the policy is calculated (Sutton et al., 1999). This direct optimization allows continuous action space and stochastic policy to be handled naturally and policy gradient methods are especially well-adapted to such problems as portfolio allocation where actions are continuous in nature (Williams, 1992).

The fundamental idea of policy gradient techniques is to optimize the policy parameters θ according to the direction that positively changes the expected cumulative reward $J(\theta)$. This is done by calculating the policy gradient $J(\theta)$ and using gradient ascent. The mathematical basis of the policy gradient theory is given by the fundamental policy gradient theorem developed by Sutton et al. (1999):

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \log \pi(a|s; \theta) \cdot Q^\pi(s, a)]$$

(2)

This term demonstrates that the gradient is contingent on the log-probability of actions under the current policy, as well as the action-value function, $Q^\pi(s, a)$, which approximates the goodness of those actions (Sutton and Barto, 2018).

Proximal Policy Optimization (PPO) is a significant breakthrough in policy gradient techniques as it mitigates the instability and sample ineffectiveness of previous algorithms. PPO can accomplish this by an extraordinarily straightforward but efficient objective function that limits the policy modifications to stay within a trust region to stop harmful massive modifications that would ruin the policy performance (Schulman et al., 2017).

The major innovation of PPO is its clipped surrogate objective function that allows several epochs of minibatch updates using the same batch of experience data. This is unlike the classical policy gradient algorithms (such as the REINFORCE) that usually only do one gradient step on each data sample, resulting in low sample efficiency (Schulman et al., 2017). PPO objective is stated as:

$$L^{CLIP}(\theta) = \widehat{E}_t \left[min(r_t(\theta)\,\widehat{A}_t,\ clip(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\,\widehat{A}_t) \right] \quad (3)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio of the new policy compared to the existing policy, $\widehat{A}_t$ is the estimate of the advantage (how much better the action taken is than the average), and ϵ is a hyperparameter (usually equal to 0.2) that controls the clipping range (Schulman et al., 2017).

The clipping mechanism operates as a way of eliminating incentives of the policy to drift too far away in its new form. At the point where the advantage is positive, the objective ceases to grow when the probability ratio rises past $1+\epsilon$, and does not permit exploiting possibly noisy advantage estimates overly aggressively. On the other hand, in the situation where the advantage is negative, the goal ceases to decline when the ratio becomes less than $1-\epsilon$ (Schulman et al., 2017). This conservative scheme makes sure that the improvement of the policy is gradual and regulated, which enhances the stability of the training by several folds in comparison to the solutions, such as Trust Region Policy Optimization (TRPO), and is much easier to execute.

PPO has shown state of the art performance in various fields, such as continuous control tasks in robotic locomotion, in playing the Atari game, and more recently as a key component of training large language models using Reinforcement Learning from Human Feedback (RLHF) (Schulman et al., 2017; Ouyang et al., 2022).

Actor-Critic Methods

The actor-critic methods constitute an intermediate approach that integrates the positive attributes of value-based and policy-based methods. These algorithms have two distinct neural networks, one an actor network which parameterizes the policy $p(a|s; \theta^P)$, and the other a "critic" network which approximates the value function $V(s; \theta^V)$ or action-value function $Q(s, a; \theta^Q)$ (Konda & Tsitsiklis, 2000). The critic gives feedback in order to update the policy of the actor so that there is less variance of policy gradient estimates without the loss of capacity to deal with continuous action spaces (Grondman et al., 2012).

The actor-critic architecture provides a very attractive tradeoff: the actor can directly optimize policies towards continuous action spaces (such as portfolio weights), but the critic can give lower-variance gradient estimates, learning a value function, and thus converges faster than pure policy gradient methods (Sutton and Barto, 2018). This two-network architecture has been especially useful in complex control problems where the sample efficiency and the ability to represent continuous actions is important.

Deep Deterministic Policy Gradient (DDPG) is an extension of the DQN framework to continuous action space, which combines deep neural networks with policy gradients (Lillicrap et al., 2015). In contrast to DQN, which gives discrete Q-values of every possible action, DDPG uses an actor-critic design in which the actor simply gives continuous action values, which makes it inherently suitable to portfolio management problems (Lillicrap et al., 2015).

DDPG may be thought of as DQN on continuous control (Lillicrap et al., 2015). It is based on the stabilization mechanisms of DQN (experience replay and target networks) but uses a deterministic policy $u(s \mid \theta^u)$ that directly takes states to particular actions, rather than action

probability distributions. This type of deterministic method is specifically beneficial in the setting where the action space is continuous, since it eliminates the computational burden of action integration when calculating policy gradients (Silver et al., 2014).

The DDPG system includes four neural networks:

1. Actor Network $\mu(s|\theta^{\mu})$: States to deterministic continuous actions.
2. Critic Network $Q(s, a \mid \theta^{Q})$: Measures the quality of state-action pairs.
3. Target Actor Network $\mu'(s|\theta^{\mu'})$: Sluggishly-updating copy of the actor.
4. Target Critic Network $Q'(s, a \mid \theta^{Q'})$: Poised version of the critic, which is updated slowly.

The critic network is also trained with the help of the temporal difference (TD) error, just like DQN, by reducing the mean squared Bellman error:

$$L(\theta^{Q}) = \mathbb{E}[(r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1}|\theta^{\mu'})|\theta^{Q'}) - Q(s_t, a_t|\theta^{Q}))^2] \quad (4)$$

To maximize the expected return, the chain rule is applied to the actor network by using the deterministic policy gradient theorem (Silver et al., 2014):

$$\nabla_{\theta\mu} J \approx \mathbb{E}[\nabla_a Q(s, a|\theta^{Q})|_{a=\mu(s)} \cdot \nabla_{\theta\mu} \mu(s|\theta^{\mu})]$$

(5)

Exploration is a crucial part of DDPG, and deterministic policies cannot do this easily. In the training, DDPG introduces noise that is temporally-correlated with the action (usually an

Ornstein-Uhlenbeck process) to the output of the actor, thus encouraging the exploration of the action space (Lillicrap et al., 2015). The noise process produces correlated perturbations which are smoother and better adapted to physical control problems and financial use than uncorrelated Gaussian noise.

DDPG also uses soft updates to update the target networks as opposed to the periodic hard updates applied in DQN. At every step in training, the target network parameters are gradually monitored towards the learned network parameters by using:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \tag{6}$$

with τ being small (usually 0.001), being the update rate (Lillicrap et al., 2015). Such continuous soft updating offers more stable learning as compared to sudden periodic updates.

DDPG has been effectively used on a variety of continuous control problems, including robotic manipulation and autonomous vehicle control (Lillicrap et al., 2015). The fact that DDPG can directly produce continuous portfolio weights through experience replay makes it an obvious tool to use in automated trading strategies in the portfolio management domain (Jiang et al., 2017). Nonetheless, DDPG is prone to hyperparameter sensitivity and overestimation bias in the Q-function, which has led to the development of later variants such as Twin Delayed DDPG (TD3) (Fujimoto et al., 2018).

2.6 Neural Network Architectures

Neural networks are grouped networks of interconnected nodes, known as artificial neurons, that are modeled after actual neurons in the human brain. One of the typical applications of neural networks is prediction. In this text, all subsequent instances of the neural networks are to Artificial Neural Networks (ANNs), except where otherwise mentioned. Various types of neural networks are listed below, some of which are referenced in the subsequent chapters:

Transformers

According to Ferrer (2024), a transformer model is a neural network that is trained on the context of sequential data and produces new data based on it. In other words, a transformer is a form of artificial intelligence that is trained to comprehend and write natural language through examining patterns of textual data on a massive scale (para. 1). Transformer models have been implemented in chatbots and text generation (IBM, n.d). Originally the transformer architecture was proposed by Google researchers in the original paper, Attention Is All You Need (Vaswani et al., 2017).

Convolutional Neural Networks

Convolutional Neural Networks are neural networks that operate using filters. They transform their original input through a series of layers, with the first layers learning low level features and basic patterns and the last layers learning more complex representations of the input. The CNNs consist of four layers, i.e., the convolutional layer, the activation layer, the pooling layer, and the fully connected layer (Google Cloud, n.d.). They are useful in the analysis of spatial information such as pictures and hence very useful in computer vision, image recognition and object detection.

Recurrent Neural Networks

Recurrent Neural Networks differ from other neural networks in the sense that they implement loops and have memory. They have the capability of remembering previous input of the past steps which is used in the analysis of context in data. RNNs operate with sequential data (such as time-series stock prices), which means it is more appropriate to the portfolio management problem.

Outlined below are some key RNN architectures:

1. **Long-Short Term Memory (LSTM) Networks**

   The more loops that the traditional RNNs use in their environment, the more they lose information in the previous iterations, and therefore, it becomes hard to connect the model to data that are separated by a long time interval in the time series. This is referred to as the **vanishing gradient problem**. LSTMs are a particular type of RNNs that are able to address this issue.

2. **Gated Recovery Units (GRUs)**

   Like LSTMs, Gated Recovery Units(GRUs) build on the RNN architecture and are built to solve the vanishing gradient problem. They are also superior to the conventional RNNs in activities such as time-series analysis. The variation between the two networks is that GRUs are more simplified - whereas LSTMs require three distinct gates (forget, input

and output) GRU only requires two gates - (update and reset). GRUs are more effective with smaller data sets whereas LSTMs are more effective with larger data sets due to the presence of the additional gate.

2.7 Deep Reinforcement Learning for Stock Market Portfolio Management

In recent years, the development of deep reinforcement learning as applied to stock market portfolio management has become especially popular due to the inherent similarity between the sequential decision-making formulation of RL and the dynamism of financial markets (Mosavi et al., 2020). The DRL agents do not need to predict future prices, which is a challenging task due to market non-stationarity and noise, but instead optimizes portfolio allocation decisions directly, taking into consideration the transaction costs, risk preferences, and evolving market regimes (Jiang et al., 2017).

Jiang et al. (2017) have shown that deep RL agents, specifically actor-critic models such as DDPG, might greatly outperform more conventional portfolio management approaches such as Buy-and-Hold and the mean-variance optimization. Their Ensemble of Identical Independent Evaluators (EIIE) algorithm that trains convolutional neural networks on historical price information has shown about four times growth of portfolios in 30 days on cryptocurrency exchanges with realistic transaction costs of 0.25% (Jiang et al., 2017).

More recent results by Filos (2019) also confirmed the efficacy of model-free RL frameworks in portfolio optimization showing a 9.2 and 13.4 percentage point better annualized cumulative returns and Sharpe ratio, respectively, over equity markets than baseline techniques. Such

findings highlight one of the primary benefits of DRA in portfolio management, namely, the possibility of adaptive strategies that adapt to market conditions without explicit financial models that can be inaccurate or fail over time as the market situation changes (Filos, 2019).

The stock market portfolio management problem is an example of a problem that can be fitted into the MDP framework: states are market observations (price histories, technical indicators, current portfolio weights), actions are portfolio rebalancing decisions (weight allocations across assets), and rewards are performance measures (such as returns or risk-adjusted returns like the Sharpe ratio) (Jiang et al., 2017; Filos, 2019). This formulation allows RL agents to acquire end-to-end trading policies that maximize the growth of a portfolio in the long term as well as risk management and reduction of transaction costs.

Regardless of the promising outcomes, various difficulties are present in the implementation of DRA to the portfolio management. Markets are non-stationary, i.e. the data distribution varies with time, as macroeconomic conditions, regulations, and behaviors of participants change (Mosavi et al., 2020). Agents based on DRL can also overfit on past data and cannot extrapolate to new environments.Also, sample efficiency and stability in training can be limited due to the scarcity of historical financial data compared to other RL fields (where unlimited synthetic experience can be generated by game simulators).

However, the model-free nature of DRL, which does not necessitate the explicit market dynamics models to learn through rewards, places the techniques in the competitive position against the traditional portfolio optimization methods. As this study shows through the systematic comparison of DQN, DDPG and PPO algorithms, the trade-offs between various

DRL architectures in terms of sample efficiency, stability and performance in portfolio management applications differ.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

The adopted methodology applied in this chapter is the Machine Learning Development Lifecycle (MLDL) methodology, a systematic and reproducible process used to create and deploy a working trading agent.

The choice of focus on model-free reinforcement learning agents is justified on theoretical grounds in the portfolio management problem; financial markets are non-stationary and inherently stochastic, and this is why it is extremely difficult to construct the correct predictive models of market behavior (Mosavi et al., 2020). Model-based methods, in which an effort is made to estimate explicitly defined transition functions of the environment, are susceptible to compounding bias when their predictions do not align with real market behaviour (Deisenroth and Rasmussen, 2011). In comparison, the model-free agents do not need the presence of an elaborate model of the environment to learn optimal policies, and thus they are better off when it comes to unpredictable financial markets (Jiang et al., 2017; Filos, 2019).

In this study, we apply and compare two state-of-the-art model-free deep reinforcement learning algorithms: Double Deep Q-Networks (DDQN) and Proximal Policy Optimization (PPO). DDQN (van Hasselt et al., 2016) addresses the over-estimation bias of ordinary DQN and remains an efficient learner through experience re-play. PPO (Schulman et al., 2017) provides better stability of the training process and has become the standard tool when it comes to

continuous control activities. Through a systematic evaluation of a value-based approach (DDQN) and a policy-gradient approach (PPO), this paper sheds more light to the performance of the various algorithm families when applied to financial portfolio optimization.

The evaluation and development process is followed in seven phases sequentially:

Problem Formulation: Mathematical model of the stock market portfolio management problem as a Markov Decision Process (MDP).

Data Acquisition: Division and gathering of historical market data.

Data Preprocessing: Feature engineering, normalisation and sequence construction.

Environment Setup: Creation of a custom OpenAI Gymnasium trading environment.

Architecture Design: DDQN and PPO agents architecture.

Model Training: Optimization of policies with proper exploration strategies.

Assessment: Strict back-testing of the built financial benchmarks.

Every implementation is based on the OpenAI Gymnasium system (Towers et al., 2023) that provides a unified interface on reinforcement learning environments and allows the simple integration of modern RL packages. Each of the methodological components is detailed in the following sections, which makes them fully reproducible and scientifically rigorous.

3.2 Dataset

The data covers 2516 trading days - 10 years of daily trading data between January 1, 2015 and December 31, 2024. To avoid data leakage and to ensure the evaluation process is fair, the data is separated into three non-overlapping sets chronologically:

Training Set: 1 January 2015 31 December 2020 (6 years, approximately 1510 days) - this training set is used to optimise the policy and update networks.

Validation Set: 1 Jan 2021 – 31 Dec 2022 (2 years, approximately 503 days) - to prevent hyper-parameter optimization and early stopping to prevent over-fitting.

Testing Set: Jan 1, 2023- Dec 1, 2024 (2 years, 503 days) - is only used to evaluate the final performance and make comparisons with baseline strategies.

We access the Yahoo Finance API through the yfinance Python library (Aroussi, 2024) to retrieve quality, adjusted close price and volume information of international equities. Within the field of studies, Yahoo Finance is the preferred option as it is a trustworthy source, reporting a great number of stocks, and it automatically accounts for splits and dividends (Sezer et al., 2020).

Our selected universe consists of 30 large tech stocks of the S&P 500 of large-cap size, according to the following criteria:

1. **Market Capitalization**: Minimum market cap of $50 billion USD to ensure liquidity and data quality
2. **Trading Volume**: Average daily volume exceeding 1 million shares to minimize the impact of illiquid trading days
3. **Data Availability**: Complete historical data from January 1, 2015 to December 31, 2024 with no missing values
4. **Sector Diversity**: Representation across software, hardware, semiconductors, and internet services to provide portfolio diversification

The most important selected assets are Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA), Alphabet (GOOGL), Amazon (AMZN), Meta (META), Tesla (TSLA) and so on.

Technology is a logical target due to volatility and growth, which is difficult to avoid, and offers RL agents a good opportunity to prove their adaptive trading abilities.

3.3 Research Environment

All implementations are developed in Python 3.10, capitalizing on its extensive ecosystem for machine learning and scientific computing. The core libraries employed include:

OpenAI Gymnasium (v0.29.1): This is an API standard that provides the standardized gym.Env interface for reinforcement learning environments, enabling seamless integration with training libraries and ensuring reproducibility (Towers et al., 2023).

PyTorch (v2.1.0): PyTorch is a deep learning framework used for neural network construction, automatic differentiation, and GPU-accelerated tensor operations (Paszke et al., 2019).

NumPy (v1.24.3) and Pandas (v2.0.3): These are essential libraries for numerical computation and data manipulation.

Matplotlib (v3.7.2) and Seaborn (v0.12.2): These libraries are used for plotting training curves, portfolio performance, and comparative analysis.

yfinance (v0.2.28): This is an interface to Yahoo Finance API for obtaining historical market data (Aroussi, 2024).

Instead of using generic simulation environments, one realizes a domain-specific custom trading environment, which subclassed gymnasium.Env. The PortfolioEnv class summarizes the Markov Decision Process dynamics of the management of a portfolio and imposes realistic trading constraints.

Initialisation of the environment is done using the pre-processed data, the rate of transaction costs and initial capital.

3.4 Model Design

3.4.1 Double Deep Q-Networks (DDQN)

Motivation and Theoretical Foundation

Normal Deep Q - Networks (DQN) overestimates the action values systematically because of the max operator in the Q-learning update (Thrun 993). It is a type of bias in overestimation that builds up in the training process and results in poor policies and unstable training (van Hasselt, 2010). To solve this problem, Double Deep Q -Networks (DDQN) proposed by van Hasselt et al. (2016) separates action selection and action evaluation into two network types.

In DDQN, the online Q -network chooses the optimal action, and the target Q -network measures the value of the action taken. The Q-learning objective is calculated as:

$$y_t = r_t + \gamma Q(s_{t+1}, \text{argmax}_{a'} Q(s_{t+1}, a'; \theta); \theta^-) \quad (7)$$

where θ represents the online network parameters and $Q(s_{t+1}, . ; \theta^-)$ represents the target network parameters. This decoupling reduces overestimation and improves learning stability (van Hasselt et al., 2016).

3.4.2 Proximal Policy Optimization (PPO)

Motivation and Theoretical Foundation

Proximal Policy Optimization (PPO) is a policy-gradient algorithm that optimises the policy directly, by gradient ascent on the expected return (Schulman et al., 2017). In contrast to

value-based techniques, PPO is inherently continuous in its action space, and so is best suited to generating portfolio weights in a non-discretised form.

The notable innovation of PPO is its clipped surrogate objective, limiting the updates of policies to be within a trust region, avoiding the disastrous large updates:

$$L^{CLIP}(\theta) = \widehat{E}_t \left[min(r_t(\theta)\, \widehat{A}_t,\ clip(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\, \widehat{A}_t) \right] \quad (8)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ is the probability ratio and $\widehat{A}_t$ is the estimated advantage.

Training stability and enabling training on different epochs of data are made possible through this mechanism that helps to enhance efficiency in samples, thereby enhancing sample efficiency (Schulman et al., 2017).

3.5 Model Training and Evaluation

3.5.1 Training Procedure

The deep deterministic Q−network (DDQN) and proximal policy optimisation (PPO) agents are executed in the Stable-Baselines3 library, which is a library that provides optimised, production-ready reinforcement learning algorithms with extensive logging and checkpointing capabilities (Raffin et al., 2021).

3.5.2 Evaluation Metrics

The effectiveness of the agents is measured with the help of conventional financial indicators:

1. Cumulative Return (CR)

Total percentage gain over the test period:

$$CR = \frac{V_T - V_0}{V_0} \times 100\% \quad (9)$$

2. Annualized Return (AR)

Geometric mean annual return:

$$AR = \left(\frac{V_T}{V_0}\right)^{\frac{252}{T}} - 1 \quad (10)$$

where 252 is the number of trading days per year.

3. Sharpe Ratio (SR)

Risk-adjusted return, measuring excess return per unit of volatility:

$$SR = \frac{\mathbb{E}[R_t - R_f]}{Std[R_t]} \quad (11)$$

where R_t is the daily return and R_f is the risk-free rate (assumed to be 0).

4. Maximum Drawdown (MDD)

Largest peak-to-trough decline in portfolio value:

$$MDD = max_{t\in[0,T]}\left(\frac{max_{s\leq t}V_s - V_t}{max_{s\leq t}V_s}\right) \quad (12)$$

5. Portfolio Turnover

Average daily portfolio rebalancing magnitude:

$$Turnover = \frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{m}|w_i(t) - w_i(t-1)| \quad (13)$$

6. Win Rate

Percentage of trading days with positive returns:

$$Win\ Rate = \frac{\#\{t: R_t > 0\}}{T} \times 100\% \quad (14)$$

3.5.3 Statistical Significance Testing

To ensure that performance differences are statistically significant rather than due to random variation, paired t-tests are conducted comparing the daily returns of RL agents versus each baseline strategy (Rice, 2006). The null hypothesis is that the mean daily returns are equal:

$$H_0 : \mu_{RL} = \mu_{baseline} \quad (15)$$

To account for multiple comparisons, the Bonferroni correction is applied.

3.5.4 Robustness Analysis

To assess the generalizability and robustness of learned policies, the following sensitivity analyses are conducted:

Transaction Cost Sensitivity: Agents are evaluated with varying transaction costs ($c \in \{0\%, 0.05\%, 0.1\%, 0.25\%, 0.5\%\}$) to examine strategy robustness under different fee structures.

Asset Universe Variation: Performance is tested on alternative portfolios (e.g., top 10 assets vs. full 30 assets) to assess scalability.

Market Regime Analysis: Separate evaluation during bull markets (positive trend), bear markets (negative trend), and high-volatility periods (COVID-19 crash in March 2020) to assess adaptability.

REFERENCES

Appel, G. (2005). Technical analysis: Power tools for active investors. Upper Saddle River, NJ: FT Press.

Aroussi, R. (2024). yfinance: Download market data from Yahoo! Finance API (Version 0.2.28) [Computer software]. Retrieved from https://github.com/ranaroussi/yfinance

Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). Deep reinforcement learning: A brief survey. IEEE Signal Processing Magazine, 34(6), 26–38. doi:10.1109/MSP.2017.2743240

Bollinger, J. (2002). Bollinger on Bollinger bands. New York, NY: McGraw-Hill.

Brownlee, J. (2020). Data preparation for machine learning: Data cleaning, feature selection, and data transforms in Python. Melbourne, Australia: Machine Learning Mastery.

Coursera. (2024, June 13). What is machine learning? A guide to its definition and types. Retrieved from https://www.coursera.org/articles/what-is-machine-learning

Deisenroth, M. P., & Rasmussen, C. E. (2011). PILCO: A model-based and data-efficient approach to policy search. In Proceedings of the 28th International Conference on Machine Learning (pp. 465–472). Madison, WI: ICML.

Ferrer, J. (2024, January 9). How transformers work: A detailed exploration of transformer architecture. DataCamp. Retrieved from https://www.datacamp.com

Filos, A. (2019). Reinforcement learning for portfolio management (MEng dissertation). Imperial College London, London, UK. Retrieved from https://arxiv.org/abs/1909.09571

François-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G., & Pineau, J. (2018). An Introduction to Deep Reinforcement Learning. Foundations and Trends in Machine Learning, 11(3–4), 219–354. doi:10.1561/2200000071

Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In Proceedings of the 35th International Conference on Machine Learning (pp. 1587–1596). Stockholm, Sweden: PMLR.

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). Sebastopol, CA: O'Reilly Media.

Google Cloud. (n.d.). What are convolutional neural networks? Retrieved from https://cloud.google.com/discover/what-are-convolutional-neural-networks

Grondman, I., Busoniu, L., Lopes, G. A., & Babuska, R. (2012). A survey of actor-critic reinforcement learning: Standard and natural policy gradients. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 42(6), 1291–1307. doi:10.1109/TSMCC.2012.2218595

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. doi:10.1162/neco.1997.9.8.1735

Hyndman, R. J., & Athanasopoulos, G. (2018). Forecasting: Principles and practice (2nd ed.). Melbourne, Australia: OTexts.

IBM. (n.d.). Transformer model. IBM Think. Retrieved from https://www.ibm.com/think/topics/transformer-model

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the 32nd International Conference on Machine Learning (pp. 448–456). Lille, France: PMLR.

Jiang, Z., Xu, D., & Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. arXiv preprint arXiv:1706.10059. Retrieved from https://arxiv.org/abs/1706.10059

Konda, V. R., & Tsitsiklis, J. N. (2000). Actor-critic algorithms. In S. A. Solla, T. K. Leen, & K. Müller (Eds.), Advances in Neural Information Processing Systems 12 (pp. 1008–1014). Cambridge, MA: MIT Press.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., . . . Wierstra, D. (2015). Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971. Retrieved from https://arxiv.org/abs/1509.02971

Markowitz, H. M. (1952). Portfolio selection. The Journal of Finance, 7(1), 77–91. doi:10.2307/2975974

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., . . . Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. doi:10.1038/nature14236

Mosavi, A., Ghamisi, P., Faghan, Y., Duan, P., Ardabili, S., & Band, S. S. (2020). Comprehensive review of deep reinforcement learning methods and applications in economics. Preprints. Retrieved from https://www.preprints.org

Murphy, J. J. (1999). Technical analysis of the financial markets: A comprehensive guide to trading methods and applications. New York, NY: New York Institute of Finance.

NVIDIA. (n.d.). Convolutional neural network (CNN). Retrieved from https://www.nvidia.com

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., . . . Lowe, R. (2022). Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155. Retrieved from https://arxiv.org/abs/2203.02155

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., . . . Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), Advances in

Neural Information Processing Systems 32 (pp. 8024–8035). Red Hook, NY: Curran Associates.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. Journal of Machine Learning Research, 22(268), 1–8. Retrieved from http://jmlr.org/papers/v22/20-1364.html

Rice, J. A. (2006). Mathematical statistics and data analysis (3rd ed.). Belmont, CA: Duxbury Press.

Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In Proceedings of the International Conference on Learning Representations (ICLR). San Juan, Puerto Rico.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. Retrieved from https://arxiv.org/abs/1707.06347

Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. Applied Soft Computing, 90, 106181. doi:10.1016/j.asoc.2020.106181

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., . . . Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484–489. doi:10.1038/nature16961

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. In Proceedings of the 31st International Conference on Machine Learning (pp. 387–395). Beijing, China: PMLR.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.).

Cambridge, MA: MIT Press.

Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In S. A. Solla, T. K. Leen, & K. Müller (Eds.), Advances in Neural Information Processing Systems 12 (pp. 1057–1063). Cambridge, MA: MIT Press.

TA-Lib. (2023). TA-Lib: Technical analysis library (Version 0.4.28) [Computer software]. Retrieved from https://ta-lib.org/

Thrun, S., & Schwartz, A. (1993). Issues in using function approximation for reinforcement learning. In Proceedings of the Fourth Connectionist Models Summer School (pp. 255–263). Hillsdale, NJ: Lawrence Erlbaum.

Towers, M., Terry, J. K., Kwiatkowski, A., Balis, J. U., Cola, G. D., Deleu, T., . . . Younis, O. G. (2023). Gymnasium. Zenodo. doi:10.5281/zenodo.8127025

van Hasselt, H. (2010). Double Q-learning. In J. D. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. S. Zemel, & A. Culotta (Eds.), Advances in Neural Information Processing Systems 23 (pp. 2613–2621). Red Hook, NY: Curran Associates.

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In Proceedings of the AAAI Conference on Artificial Intelligence, 30(1), 2094–2100. doi:10.1609/aaai.v30i1.10295

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., . . . Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), Advances in Neural Information Processing Systems 30 (pp. 5998–6008). Red Hook, NY: Curran Associates.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling

network architectures for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning (pp. 1995–2003). New York, NY: PMLR.

Wilder, J. W. (1978). New concepts in technical trading systems. Greensboro, NC: Trend Research.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4), 229–256. doi:10.1007/BF00992696